

Technological University Of Dublin

INVENTORY MANAGEMENT API

B00150058 – Daniel Aigbe



Lecturer: Kyle Goslin

Date: 02/04/2026

Introduction.....	2
Technologies Used:	3
FastAPI Application Code:	3
Main API File (app.py):	4
Database Configuration (database.py)	7
Data Model (models.py).....	7
CSV Upload Script (load_csv.py)	8
Requirements.txt	8
generate_readme.py	9
API Endpoints (Swagger + Testing)	11
Swagger UI	11
Get All Products	12
Get Single Product	13
Add New Product	14
Delete Product.....	15
Starts With Letter	16
Pagination	17
Currency Conversion.....	19
Unit Testing (Newman / Postman).....	20
Postman Collection	21
Newman Execution	22
Test Results.....	22
All Tests Passed	22
Docker Configuration	22
Dockerfile (CODE).....	22
Docker Build.....	22

Docker finished building:	22
Image being built successfully:	22
Docker Run:.....	22
Container Running:	22
Jenkins Pipeline	22
Jenkins Pipeline Stages.....	22
Jenkins Execution.....	22
Pipeline Output.....	22
Figure 1: GitHub checkout output.....	22
Figure 2: Tool check output.....	22
Figure 3: README generation output	22
Figure 4: Docker build output	22
Figure 6: API startup check	22
Figure 7: Newman test execution	22
Figure 8: ZIP creation output	22
Figure 9: Cleanup and final success	22
Final Build Artifacts (ZIP File)	22
API Performance Monitoring.....	22
Conclusion	22
References	22
Documentation (supporting sources)	22

Introduction

This project presents the design and implementation of an Inventory Management API developed using FastAPI. The system provides structured access to product data through clearly defined endpoints that handle operations such as retrieving all products, fetching a single product by ID, adding new entries, deleting existing records, filtering results based on specific criteria, paginating datasets, and converting product prices. The API returns consistent JSON responses and uses appropriate HTTP status codes to indicate successful and unsuccessful requests, while input validation ensures that all data follows the required format. Testing was carried out using Postman and Newman, confirming that all endpoints function correctly and return accurate results. Docker was used to containerise the application, ensuring consistent execution across environments and simplifying deployment. A Jenkins pipeline was implemented to automate the workflow, including building, testing, and packaging the application. Monitoring was also applied to track uptime and response performance, with results showing stable availability and consistent response times. This project demonstrates a complete backend development lifecycle, integrating API development, testing, containerisation, automation, and monitoring into a single, reliable system.

Technologies Used:

This project uses several core technologies to support development, testing, deployment, and monitoring. FastAPI serves as the main backend framework, providing high performance and automatic validation of requests. Python is used as the primary

programming language to implement all application logic. MongoDB is used as the database to store product data in a flexible and scalable format. Uvicorn runs the FastAPI application as an ASGI server, enabling efficient handling of requests. Postman is used to test API endpoints manually, while Newman executes the same tests automatically through the command line. Docker is used to containerise the application, ensuring consistent behaviour across different environments. Jenkins is used to automate the pipeline, including building the application, running tests, and generating final outputs. Monitoring tools are used to track uptime and response time, ensuring the API remains available and performs consistently.

FastAPI Application Code:

Main API File (app.py):

```
from fastapi import FastAPI, HTTPException, Query
from database import products_collection
from models import Product
import requests

app = FastAPI(title="Inventory Management API")

# -----
# Helper function
```

```

# -----
def product_helper(product):
    return {
        "ProductID": product["ProductID"],
        "Name": product["Name"],
        "UnitPrice": product["UnitPrice"],
        "StockQuantity": product["StockQuantity"],
        "Description": product.get("Description", "")
    }

# -----
# 1. GET SINGLE PRODUCT
# -----
@app.get("/getSingleProduct", tags=["Products"])
def get_single_product(
    id: int = Query(..., gt=0, description="Product ID must be > 0")
):
    product = products_collection.find_one({"ProductID": id})
    if product:
        return product_helper(product)
    raise HTTPException(status_code=404, detail="Product not found")

# -----
# 2. GET ALL PRODUCTS
# -----
@app.get("/getAll", tags=["Products"])
def get_all():
    products = products_collection.find()
    return [product_helper(p) for p in products]

# -----
# 3. ADD NEW PRODUCT
# -----
@app.post("/addNew", tags=["Products"])
def add_new(product: Product):
    existing = products_collection.find_one({"ProductID": product.ProductID})
    if existing:
        raise HTTPException(status_code=400, detail="Product already exists")

    products_collection.insert_one(product.dict())
    return {"message": "Product added successfully"}

```

```

# -----
# 4. DELETE PRODUCT
# -----
@app.delete("/deleteOne", tags=["Products"])
def delete_one(
    id: int = Query(..., gt=0, description="Product ID must be > 0")
):
    result = products_collection.delete_one({"ProductID": id})
    if result.deleted_count == 1:
        return {"message": "Product deleted successfully"}
    raise HTTPException(status_code=404, detail="Product not found")

# -----
# 5. STARTS WITH LETTER
# -----
@app.get("/startsWith", tags=["Products"])
def starts_with(
    letter: str = Query(..., min_length=1, max_length=1, description="Single
letter")
):
    results = products_collection.find({
        "Name": {"$regex": f"^{letter}", "$options": "i"}
    })
    return [product_helper(p) for p in results]

# -----
# 6. PAGINATE (batch of 10)
# -----
@app.get("/paginate", tags=["Products"])
def paginate(
    start_id: int = Query(..., gt=0),
    end_id: int = Query(..., gt=0)
):
    if start_id > end_id:
        raise HTTPException(status_code=400, detail="start_id must be <= end_id")

    results = products_collection.find({
        "ProductID": {"$gte": start_id, "$lte": end_id}
    }).sort("ProductID", 1).limit(10)

```

```

        return [product_helper(p) for p in results]

# -----
# 7. CONVERT USD → EUR
# -----
@app.get("/convert", tags=["Products"])
def convert_price(
    id: int = Query(..., gt=0)
):
    product = products_collection.find_one({"ProductID": id})
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")

    try:
        response = requests.get("https://api.exchangerate-api.com/v4/latest/USD",
                                timeout=5)
        if response.status_code != 200:
            raise Exception("API error")

        data = response.json()
        eur_rate = data["rates"]["EUR"]

    except Exception:
        raise HTTPException(status_code=500, detail="Currency API unavailable")

    converted_price = product["UnitPrice"] * eur_rate

    return {
        "ProductID": id,
        "PriceUSD": product["UnitPrice"],
        "PriceEUR": round(converted_price, 2)
    }

```

Database Configuration (database.py)

```
import os
from pymongo import MongoClient
MONGO_URL = os.getenv(
    "MONGO_URL",
    "mongodb+srv://admin11:adminpass11@inventory-management-
db.ue0pccy.mongodb.net/inventoryDB?retryWrites=true&w=majority"
)
client = MongoClient(MONGO_URL)
db = client["inventoryDB"]
products_collection = db["products"]
```

Data Model (models.py)

```
from pydantic import BaseModel

class Product(BaseModel):
    ProductID: int
    Name: str
    UnitPrice: float
    StockQuantity: int
    Description: str
```

CSV Upload Script (load_csv.py)

```
import os
import pandas as pd
from pymongo import MongoClient

MONGO_URL = os.getenv(
    "MONGO_URL",
```

```
    "mongodb+srv://admin11:adminpass11@inventory-management-  
db.ue0pccy.mongodb.net/inventoryDB?retryWrites=true&w=majority"  
)  
  
client = MongoClient(MONGO_URL)  
db = client["inventoryDB"]  
collection = db["products"]  
  
df = pd.read_csv("products.csv")  
data = df.to_dict(orient="records")  
  
if data:  
    collection.insert_many(data)  
print("CSV uploaded successfully")
```

Requirements.txt

```
fastapi==0.135.2  
uvicorn==0.42.0  
pymongo==4.16.0  
pandas==2.3.3  
numpy==2.2.6  
requests==2.32.3  
pydantic==2.12.5  
pytest==8.3.4
```

generate_readme.py

```
from datetime import datetime

content = f"""Inventory Management API - Auto Generated README
Generated: {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}

FastAPI Documentation:
http://localhost:8000/docs

API Endpoints:

1. /getSingleProduct
   Method: GET
   Parameters:
   - id (integer, required)
   Example:
   /getSingleProduct?id=1001

2. /getAll
   Method: GET
   Parameters:
   - None
   Example:
   /getAll

3. /addNew
   Method: POST
   Body (JSON):
   {{
     "ProductID": 999,
     "Name": "Test Item",
     "UnitPrice": 10.5,
     "StockQuantity": 20,
```

```
        "Description": "Test product"
    }}
}}
```

4. /deleteOne

Method: DELETE

Parameters:

- id (integer, required)

Example:

/deleteOne?id=999

5. /startsWith

Method: GET

Parameters:

- letter (string, required, single character)

Example:

/startsWith?letter=s

6. /paginate

Method: GET

Parameters:

- start_id (integer, required)

- end_id (integer, required)

Example:

/paginate?start_id=1001&end_id=1025

7. /convert

Method: GET

Parameters:

- id (integer, required)

Example:

/convert?id=1007

Tech Stack:

- FastAPI
- MongoDB
- Docker
- Jenkins
- Newman
- Pytest

"""

```
with open("README.txt", "w", encoding="utf-8") as f:
    f.write(content)
```

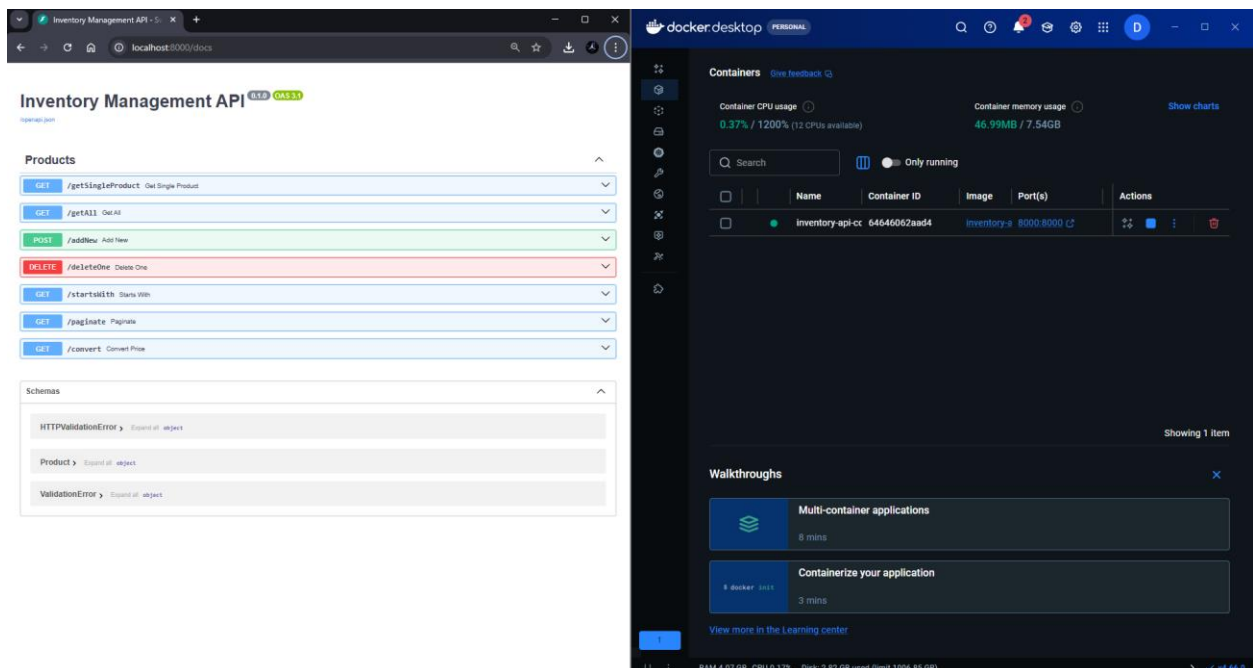
```
print("README.txt generated successfully")
```

API Endpoints (Swagger + Testing)

The API endpoints were tested using both Swagger UI and Postman. Swagger provides an interactive interface for sending requests and viewing responses directly from the browser. Postman was used to validate functionality and confirm that each endpoint returned the correct response format and status codes. All endpoints returned status code 200 during successful execution and produced structured JSON responses.

Swagger UI

Swagger UI provides a visual interface for interacting with all available API endpoints. It displays each route, required parameters, and expected responses. This allows quick verification of functionality without requiring external tools. The interface confirms that all endpoints are correctly registered and accessible.



Get All Products

This endpoint retrieves all products stored in the database. The response returns an array of product objects. Each object contains ProductID, Name, UnitPrice, StockQuantity, and Description.

The results confirm that:

- The dataset is returned correctly
- Multiple products are present
- The structure follows JSON format

The screenshot shows a REST client interface with the following sections:

- Products** (header)
- GET /getSingleProduct** (selected endpoint)
- Parameters** (No parameters)
- Execute** (button)
- Responses** (tab)
- Curl** (curl -X 'GET' -H 'accept: application/json' -H 'http://localhost:8000/getAll')
- Request URL** (http://localhost:8000/getAll)
- Server response** (200)
- Response body** (JSON array of 5 products)
- Response headers** (empty)

The JSON response body is as follows:

```
{
  "ProductID": 1001,
  "Name": "NVIDIA RTX 4090",
  "UnitPrice": 1599.99,
  "StockQuantity": 12,
  "Description": "High-end GPU with 24GB VRAM for 4K gaming."
},
{
  "ProductID": 1002,
  "Name": "AMD Ryzen 9 7950X",
  "UnitPrice": 549,
  "StockQuantity": 25,
  "Description": "16-core 32-thread enthusiast desktop processor."
},
{
  "ProductID": 1003,
  "Name": "Samsung 990 Pro 2TB",
  "UnitPrice": 129.99,
  "StockQuantity": 40,
  "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
},
{
  "ProductID": 1004,
  "Name": "Corsair Vengeance 32GB DDR5",
  "UnitPrice": 115,
  "StockQuantity": 60,
  "Description": "6000MHz CL36 Dual Channel Memory Kit."
},
{
  "ProductID": 1005,
  "Name": "Apple M3 Pro 16GB",
  "UnitPrice": 1499,
  "StockQuantity": 8,
  "Description": "Latest Apple silicon laptop chip with 16GB unified memory."
}
```

Get Single Product

This endpoint retrieves a specific product using its ProductID. The response contains a single JSON object representing the selected product.

The results confirm that:

- The correct product is returned
- The ProductID matches the request
- The response structure is valid

Products

GET

/getSingleProduct

Get Single Product

Parameters

Cancel

Name	Description
id * required integer (query)	Product ID must be > 0 1001

Execute

Clear

Responses

Curl

```
curl -X 'GET' \  
  'http://127.0.0.1:8000/getSingleProduct?id=1001' \  
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/getSingleProduct?id=1001
```

Server response

Code	Details
200	Response body <pre>{ "ProductID": 1001, "Name": "NVIDIA RTX 4090", "UnitPrice": 1599.99, "StockQuantity": 12, "Description": "High-end GPU with 24GB VRAM for 4K gaming." }</pre> Download

Add New Product

This endpoint allows insertion of a new product into the database. The request includes product details, and the response confirms successful addition.

The results confirm that:

- The product is added successfully
- The API returns a success message
- The status code is correct

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "ProductID": 999,
  "Name": "Test Item",
  "UnitPrice": 10.5,
  "StockQuantity": 20,
  "Description": "Test product1"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/addNew' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "ProductID": 999,
  "Name": "Test Item",
  "UnitPrice": 10.5,
  "StockQuantity": 20,
  "Description": "Test product1"
}'
```

Request URL

```
http://127.0.0.1:8000/addNew
```

Server response

Delete Product

This endpoint allows insertion of a new product into the database. The request includes product details, and the response confirms successful addition.

The results confirm that:

- The product is added successfully
- The API returns a success message
- The status code is correct

DELETE /deleteOne Delete One

Parameters

Cancel

Name	Description
id * required integer (query)	Product ID must be > 0

1005

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/deleteOne?id=1005' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/deleteOne?id=1005
```

Server response

Code	Details
200	Response body <pre>{ "message": "Product deleted successfully" }</pre> Download Response headers <pre>content-length: 42 content-type: application/json date: Thu, 26 Mar 2026 08:03:52 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Starts With Letter

This endpoint filters products based on the first letter of the product name. The response returns matching products only.

The results confirm that:

- All returned names start with the specified letter
- Filtering works correctly
- The response format is valid

letter

string

(query)

Single letter

S

minLength: 1

maxLength: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/startsWith?letter=s' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/startsWith?letter=s
```

Server response

Code	Details
200	Response body <pre>[{ "ProductID": 1003, "Name": "Samsung 990 Pro 2TB", "UnitPrice": 179.99, "StockQuantity": 40, "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads." }, { "ProductID": 1016, "Name": "Seasonic Focus GX-1000", "UnitPrice": 169, "StockQuantity": 20, "Description": "1000W 80+ Gold power supply for high-end builds." }, { "ProductID": 1029, "Name": "SteelSeries Apex Pro", "UnitPrice": 179.99, "StockQuantity": 20, "Description": "Mechanical keyboard with adjustable actuation." }, { "ProductID": 1039, "Name": "Seagate IronWolf 8TB", "UnitPrice": 189.99, "StockQuantity": 15, </pre>

Download

Pagination

This endpoint returns products within a specified ID range. The response is limited to a maximum of 10 results.

The results confirm that:

- Returned products fall within the range
- The number of results is limited
- Pagination logic works correctly

Name	Description
start_id * required integer (query)	1001
end_id * required integer (query)	1025
Execute Clear	

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/paginate?start_id=1001&end_id=1025' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/paginate?start_id=1001&end_id=1025
```

Server response

Code	Details
200	Response body <pre>[{ "ProductID": 1001, "Name": "NVIDIA RTX 4090", "UnitPrice": 1599.99, "StockQuantity": 12, "Description": "High-end GPU with 24GB VRAM for 4K gaming." }, { "ProductID": 1002, "Name": "AMD Ryzen 9 7950X", "UnitPrice": 549, "StockQuantity": 25, "Description": "16-core 32-thread enthusiast desktop processor." }, { "ProductID": 1003, "Name": "Samsung 990 Pro 2TB", "UnitPrice": 179.99, "StockQuantity": 40, "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads." }]</pre>

http://127.0.0.1:8000/paginate?start_id=1001&end_id=1025

Server response

Code Details

200

Response body

```
},
{
  "ProductID": 1004,
  "Name": "Corsair Vengeance 32GB DDR5",
  "UnitPrice": 115,
  "StockQuantity": 60,
  "Description": "6000MHz CL36 Dual Channel Memory Kit."
},
{
  "ProductID": 1006,
  "Name": "EVGA SuperNova 850 G6",
  "UnitPrice": 139.5,
  "StockQuantity": 30,
  "Description": "850W 80 Plus Gold Fully Modular Power Supply."
},
{
  "ProductID": 1007,
  "Name": "NZXT H7 Flow Case",
  "UnitPrice": 129.99,
  "StockQuantity": 22,
  "Description": "Mid-tower ATX case with high-airflow mesh front."
},
{
  "ProductID": 1008,
  "Name": "Noctua NH-D15",
  "UnitPrice": 109.95,
  "StockQuantity": 45,
  "Description": "Dual-tower flagship CPU air cooler."
}
```



Download

Response headers

```
content-length: 1447
content-type: application/json
date: Thu, 26 Mar 2026 08:09:31 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

"string"

Currency Conversion

This endpoint converts product prices from USD to EUR. The response includes both values.

The results confirm that:

- USD price is retrieved correctly
- EUR value is calculated correctly
- Response contains both fields

GET

/convert

Convert Price

^

Parameters

Cancel

Name	Description
id * required integer (query)	1007

Execute

Clear

Responses

Curl

```
curl -X 'GET' \  
  'http://127.0.0.1:8000/convert?id=1007' \  
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/convert?id=1007
```

Server response

Code	Details
200	Response body <pre>{ "ProductID": 1007, "PriceUSD": 129.99, "PriceEUR": 112.31 }</pre> Download Response headers <pre>content-length: 54 content-type: application/json date: Thu, 26 Mar 2026 08:08:26 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Unit Testing (Newman / Postman)

This section verifies the correctness and reliability of all API endpoints through structured testing. Testing was performed using Postman for individual validation and Newman for automated command line execution. Each endpoint includes assertions to confirm correct behaviour, response format, and expected data.

Postman Collection

The Postman collection contains all endpoints, including Get All, Get Single Product, Add New Product, Delete Product, Starts With, Pagination, and Convert Price. Each request includes multiple assertions to validate functionality and data integrity. The results show that all endpoints return correct responses and meet the expected conditions defined in the test scripts.

InventoryAPI > Convert Price

GET http://localhost:8000/convert?id=1008

Send

Pre-request

Post-response

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Response is JSON", function () {
6   pm.response.to.be.json;
7 });
8
9 pm.test("Delete successful", function () {
10   const data = pm.response.json();
11   pm.expect(data).to.have.property("message");
12 });
```

Body Cookies Headers 4 Test Results 5/5 200 OK - 706 ms - 177 B Save Response

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#) Run Collection

Filter Results

- PASSED Status code is 200
- PASSED Response is JSON
- PASSED Has ProductID
- PASSED Has USD price
- PASSED Has EUR price

ENVIRONMENTS

SPECS

FLAWS

InventoryAPI > Delete Product

DELETE http://localhost:8000/deleteOne?id=1010

Send

Pre-request

Post-response

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Response is JSON", function () {
6   pm.response.to.be.json;
7 });
8
9 pm.test("Delete successful", function () {
10   const data = pm.response.json();
11   pm.expect(data).to.have.property("message");
12 });
```

Body Cookies Headers 4 Test Results 3/3 200 OK - 33 ms - 167 B Save Response

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#) Run Collection

Filter Results

- PASSED Status code is 200
- PASSED Response is JSON
- PASSED Delete successful

ENVIRONMENTS

SPECS

FLAWS

InventoryAPI > Get All

GET http://localhost:8000/getAll

Pre-request

Post-response

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Response is JSON", function () {
6   pm.response.to.be.json;
7 });
8
9 pm.test("Response is an array", function () {
10   const data = pm.response.json();
11   pm.expect(data).to.be.an("array");
12 });
13
14 pm.test("Array is not empty", function () {
15   const data = pm.response.json();
16   pm.expect(data.length).to.be.above(0);
17 });
```

Body Cookies Headers 4 Test Results 4/4 200 OK · 63 ms · 27.47 KB

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#)

Filter Results

- PASSED Status code is 200
- PASSED Response is JSON
- PASSED Response is an array
- PASSED Array is not empty

InventoryAPI > getSingleProduct

GET http://localhost:8000/getSingleProduct?id=1001

Pre-request

Post-response

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Response is JSON", function () {
6   pm.response.to.be.json;
7 });
8
9 pm.test("Has ProductID", function () {
10   const data = pm.response.json();
11   pm.expect(data).to.have.property("ProductID");
12 });
13
14 pm.test("Correct product returned", function () {
15   const data = pm.response.json();
16   pm.expect(data.ProductID).to.eql(1001);
17 });
```

Body Cookies Headers 4 Test Results 4/4

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#)

Filter Results

- PASSED Status code is 200
- PASSED Response is JSON
- PASSED Has ProductID
- PASSED Correct product returned

InventoryAPI > Paginate

GET http://localhost:8000/paginate?start_id=1001&end_id=1025

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
1 pm.response.to.have.status(200);
2
3
4
5 pm.test("Response is JSON", function () {
6   pm.response.to.be.json;
7 });
8
9 pm.test("Response is array", function () {
10   const data = pm.response.json();
11   pm.expect(data).to.be.an("array");
12 });
13
14 pm.test("Max 10 results", function () {
15   const data = pm.response.json();
16   pm.expect(data.length).to.be.at.most(10);
17 });
```

Body Cookies Headers 4 Test Results 4/4 200 OK 51 ms 1.54 KB

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#)

Filter Results

- PASSED Status code is 200
- PASSED Response is JSON
- PASSED Response is array
- PASSED Max 10 results

ENVIRONMENTS

SPECS

InventoryAPI > Add New Product

POST http://localhost:8000/addNew

Docs Params Authorization Headers 8 Body Scripts Settings

Pre-request

Post-response

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Response is JSON", function () {
6   pm.response.to.be.json;
7 });
8
9 pm.test("Product added successfully", function () {
10   const data = pm.response.json();
11   pm.expect(data).to.have.property("message");
12 });
```

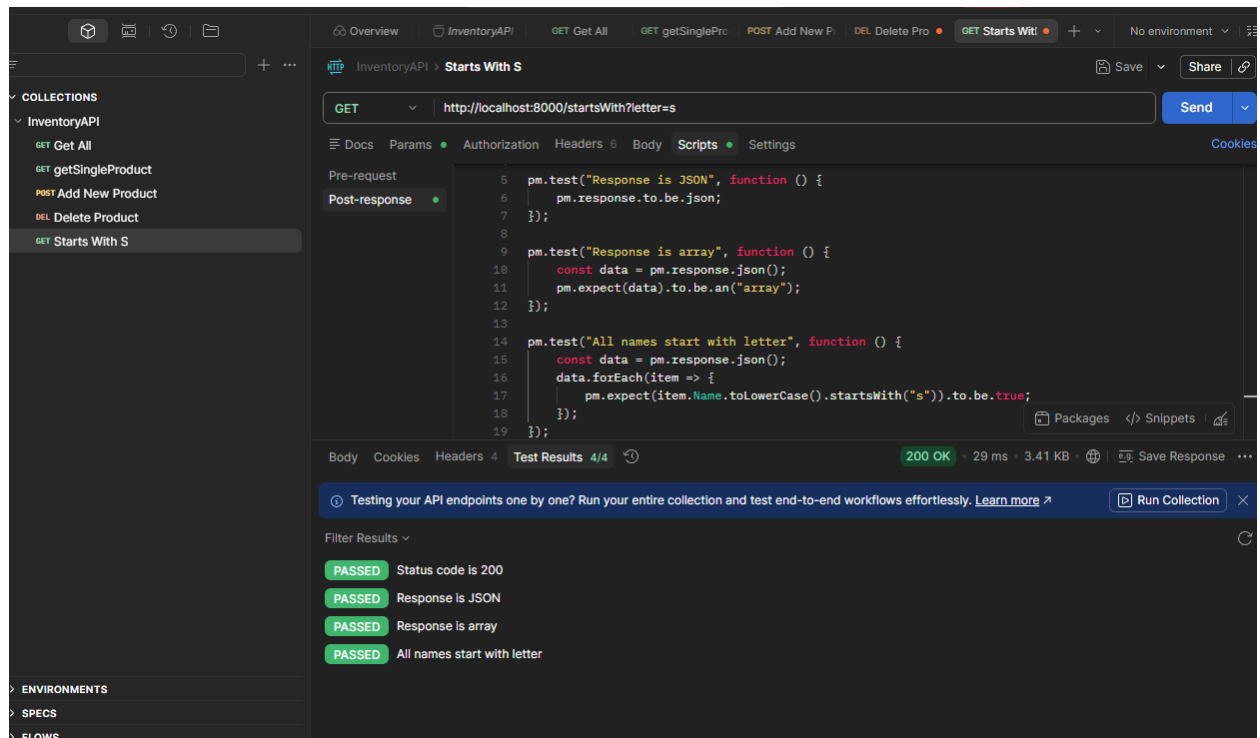
Body Cookies Headers 4 Test Results 3/3 200 OK 90 ms 165 B

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#)

Filter Results

- PASSED Status code is 200
- PASSED Response is JSON
- PASSED Product added successfully

ENVIRONMENTS



Newman Execution

The Postman collection was executed using Newman through the command line. This step ensures that all tests run automatically without manual interaction. The execution simulates real workflow conditions and validates the API in a repeatable environment.

The command line execution processes all requests and runs all associated assertions. This confirms that the API performs consistently across multiple runs.

```
→ Get All
GET http://localhost:8001/getAll [200 OK, 27.98kB, 207ms]
✓ Status code is 200
✓ Response is JSON
✓ Response is an array
✓ Array is not empty

→ getSingleProduct
GET http://localhost:8001/getSingleProduct?id=1001 [200 OK, 267B, 15ms]
✓ Status code is 200
✓ Response is JSON
✓ Has ProductID
✓ Correct product returned

→ Add New Product
POST http://localhost:8001/addNew [200 OK, 165B, 40ms]
✓ Status code is 200
✓ Response is JSON
✓ Product added successfully

→ Delete Product
DELETE http://localhost:8001/deleteOne?id=9997 [200 OK, 167B, 27ms]
✓ Status code is 200
✓ Response is JSON
✓ Delete successful

→ Starts With S
GET http://localhost:8001/startsWith?letter=s [200 OK, 3.49kB, 16ms]
✓ Status code is 200
✓ Response is JSON
✓ Response is array
✓ All names start with letter

→ Paginate
GET http://localhost:8001/paginate?start_id=1001&end_id=1025 [200 OK, 1.57kB, 17ms]
✓ Status code is 200
✓ Response is JSON
✓ Response is array
✓ Max 10 results

→ Convert Price
GET http://localhost:8001/convert?id=1008 [200 OK, 178B, 847ms]
✓ Status code is 200
✓ Response is JSON
✓ Has ProductID
✓ Has USD price
✓ Has EUR price
```

Test Results

The Newman output provides a detailed summary of the execution. The results include total requests, number of assertions, execution time, and response performance metrics. The output confirms that all tests completed successfully with no failures recorded.

The results show:

- All endpoints returned status code 200
- All responses were in JSON format
- All assertions passed successfully
- No failed tests were recorded
- Response times remained stable across all requests

	executed	failed
iterations	1	0
requests	7	0
test-scripts	7	0
prerequest-scripts	0	0
assertions	27	0
total run duration: 1754ms		
total data received: 32.94kB (approx)		
average response time: 167ms [min: 15ms, max: 847ms, s.d.: 284ms]		

```
(.venv) C:\inventory mgmt api>uvicorn app:app --reload --port 8001
INFO: Will watch for changes in these directories: ['C:\\inventory mgmt api']
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO: Started reloader process [12476] using StatReload
INFO: Started server process [21548]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:55896 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:55896 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:63055 - "GET /getAll HTTP/1.1" 200 OK
INFO: 127.0.0.1:63055 - "GET /getSingleProduct?id=1001 HTTP/1.1" 200 OK
INFO: 127.0.0.1:63055 - "POST /addNew HTTP/1.1" 200 OK
INFO: 127.0.0.1:63055 - "DELETE /deleteOne?id=9997 HTTP/1.1" 200 OK
INFO: 127.0.0.1:63055 - "GET /startsWith?letter=s HTTP/1.1" 200 OK
INFO: 127.0.0.1:63055 - "GET /paginate?start_id=1001&end_id=1025 HTTP/1.1" 200 OK
INFO: 127.0.0.1:63055 - "GET /convert?id=1008 HTTP/1.1" 200 OK
```

All Tests Passed

The testing phase confirms that the API meets all functional requirements. Every endpoint behaves correctly, returns valid data, and passes all defined test conditions. The use of Postman and Newman ensures that the system is reliable and ready for integration into automated pipelines.

Docker Configuration

Docker was used to containerise the API to ensure consistent execution across different environments. Containerisation removes dependency issues and allows the application to run reliably on any system with Docker installed. The API and all required files are packaged into a single image, which can then be executed as a container.

Dockerfile (CODE)

This Dockerfile sets up a container to run the FastAPI application. It starts with an Ubuntu base image and installs Python, pip, and required tools. The project dependencies are installed from the requirements file, and all application files are copied into the container.

Port 8000 is opened to allow access to the API. The final command runs the application using Uvicorn so the API starts automatically when the container runs.

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y \
    python3 \
    python3-pip \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Docker Build

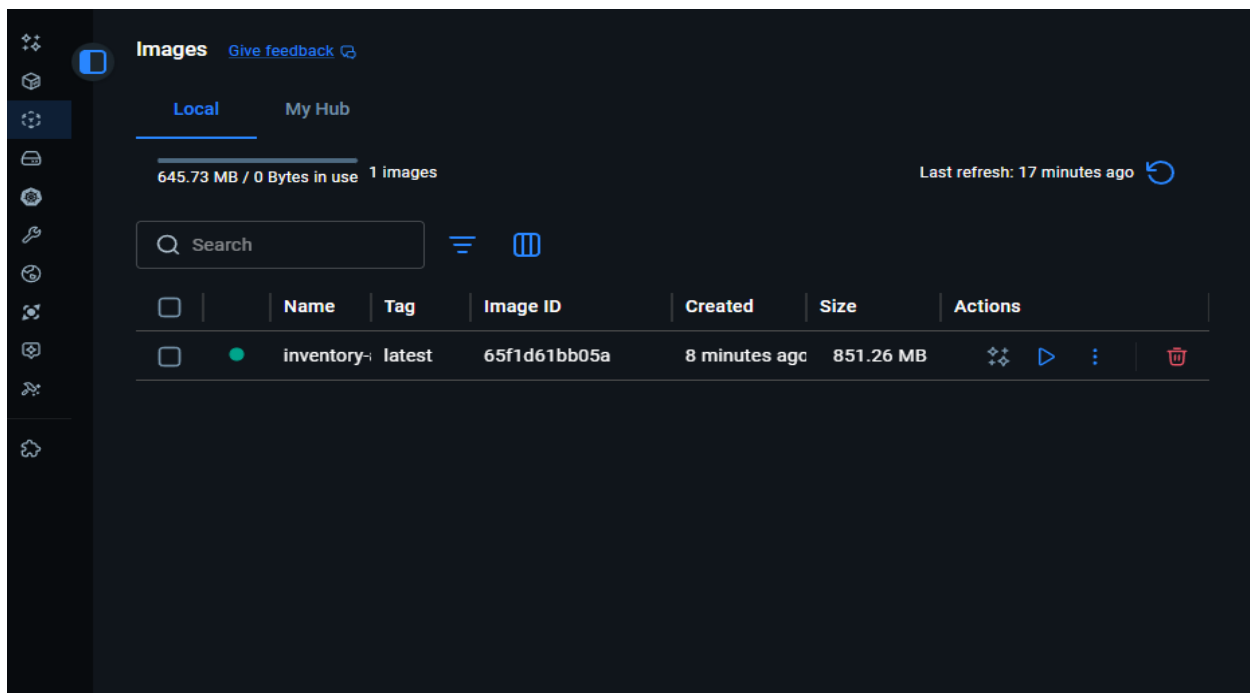
The Docker image was built using the Dockerfile. This step packages the application and all dependencies into a reusable image.

The build process completed successfully, confirming that all dependencies were installed correctly and that the image was created without errors.

Docker finished building:

```
C:\inventory mgmt api>docker build -t inventory-api .  
[+] Building 58.6s (11/11) FINISHED  
ker:desktop-linux  
=> [internal] load build definition from dockerfile  
0.0s  
=> => transferring dockerfile: 469B  
0.0s
```

Image being built successfully:



Docker Run:

The Docker container was started using the built image. The container runs the FastAPI application and exposes it through a defined port.

The container was configured to map port 8001 on the host system to port 8000 inside the container. This allows access to the API through a browser or testing tools.

The execution confirms that the container starts successfully and runs in the background.

This is the command I used to run the docker container:

```
11.45  
C:\inventory mgmt api>docker run -d --name inventory-api-container -p 8000:8000 inventory-api  
64646062aad43e5b961b99a2df686e90cae577e1109bed1e9f58219272fef8c  
C:\inventory mgmt api>
```

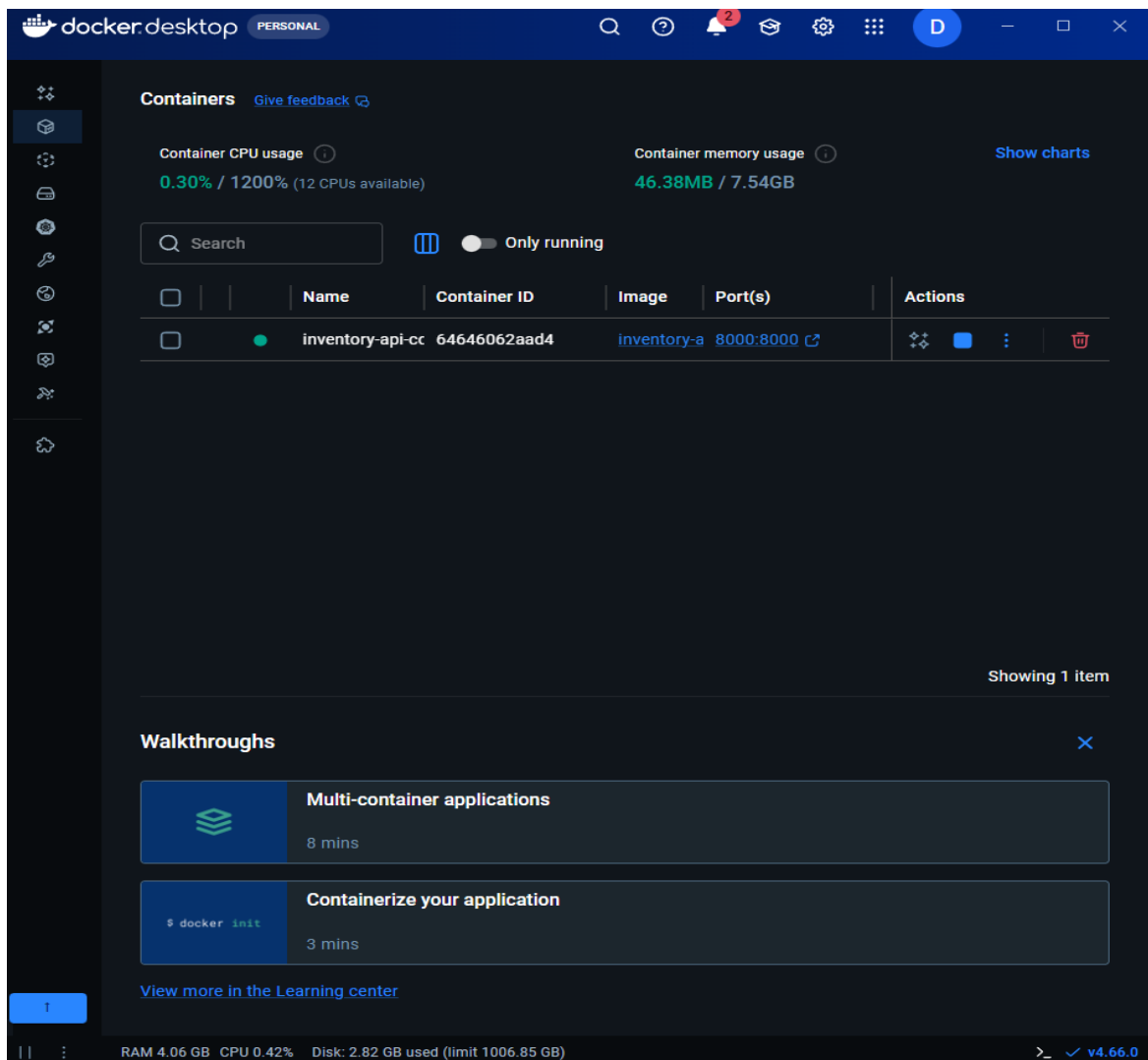
Container Running:

The Docker container was started using the built image. The container runs the FastAPI application and exposes it through a defined port.

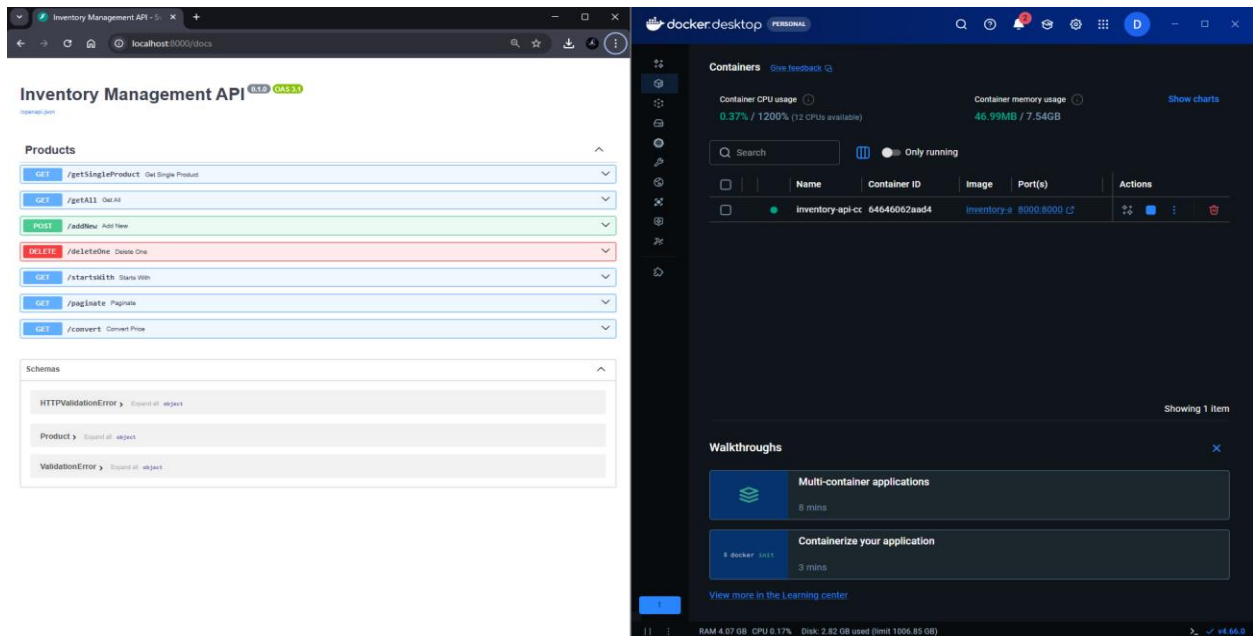
The container was configured to map port 8001 on the host system to port 8000 inside the container. This allows access to the API through a browser or testing tools.

The execution confirms that the container starts successfully and runs in the background.

Docker running here:



Swagger running with docker here:



Jenkins Pipeline

The Jenkins pipeline was used to automate the full build, test, and packaging workflow for the Inventory Management API. This removed the need to run each step manually and ensured that the same process was followed every time the project was executed. The pipeline handled code retrieval, dependency checks, Docker image creation, container execution, automated testing, ZIP generation, and cleanup.

Jenkins Pipeline Stages

The pipeline was divided into a number of stages. First, Jenkins checked out the project from the GitHub repository. It then verified that the required tools were available, including Python, Docker, Node.js, and Newman. After this, a README file was generated automatically. The pipeline then built the Docker image for the API and started a container using port 8001 to avoid conflicts with port 8000 already in use on the system. Once the API server became available, Newman executed the Postman collection against the running container. After all tests passed, Jenkins created the final ZIP file and removed the running Docker container.

Jenkins Execution

The Jenkins execution confirmed that the entire workflow completed successfully from start to finish. The logs showed that the repository was pulled from GitHub, the Docker

image was built without errors, and the container started successfully. The health check confirmed that the API documentation page at <http://localhost:8001/docs> was reachable before the automated tests were started.

 **Jenkins** / InventoryAPI-Pipeline

Status

</> Changes

▶ Build Now

⚙️ Configure

🗑️ Delete Pipeline

📁 Stages

✎ Rename

❓ Pipeline Syntax

✓ **InventoryAPI-Pipeline**

📦 Last Successful Artifacts

📄 complete-2026-03-30-16-13-46.zip	5.96 KiB	👁️ view
📄 complete-2026-04-01-18-32-23.zip	5.93 KiB	👁️ view
📄 README.txt	1.16 KiB	👁️ view

Permalinks

- [Last build \(#13\), 5 min 7 sec ago](#)
- [Last stable build \(#13\), 5 min 7 sec ago](#)
- [Last successful build \(#13\), 5 min 7 sec ago](#)
- [Last failed build \(#12\), 28 min ago](#)
- [Last unsuccessful build \(#12\), 28 min ago](#)
- [Last completed build \(#13\), 5 min 7 sec ago](#)

Builds > ...

🔍 Filter /

Today

✓ #13 6:31 PM ▼

✗ #12 6:08 PM ▼

✗ #11 6:05 PM ▼

March 30, 2026

✓ #10 4:13 PM ▼

Pipeline Output

The Jenkins pipeline output was captured to show each major stage of the automated workflow. The screenshots provide evidence that the pipeline successfully retrieved the project from GitHub, checked required tools, generated the README file, built the Docker image, started the API container, verified application startup, executed Newman tests, created the final ZIP artifact, and removed the running container after completion.

Together, these outputs confirm that the pipeline fulfilled all required tasks in the correct sequence

Figure 1: GitHub checkout output

Console

```
Started by user Daniel Aigbe
Obtained Jenkinsfile from git https://github.com/dannya123-blm/inventory-mgmt-api.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git.exe
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/dannya123-blm/inventory-mgmt-api.git # timeout=10
Fetching upstream changes from https://github.com/dannya123-blm/inventory-mgmt-api.git
> git.exe --version # timeout=10
> git --version # 'git version 2.53.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/dannya123-blm/inventory-mgmt-api.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision cc9cd6d84b88ff794b20acd56c57a42191031571 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f cc9cd6d84b88ff794b20acd56c57a42191031571 # timeout=10
Commit message: "changed json file for collection and changed url in postman to base url for 8001 port connections because jenkins ports were colliding on port 8000"
> git.exe rev-list --no-walk c7314b7549c2ed0b50bb89059506be72e7b0f792 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout from GitHub)
[Pipeline] checkout
The recommended git tool is: git.exe
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline\.git # timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/dannya123-blm/inventory-mgmt-api.git # timeout=10
Fetching upstream changes from https://github.com/dannya123-blm/inventory-mgmt-api.git
> git.exe --version # timeout=10
> git --version # 'git version 2.53.0.windows.1'
> git.exe fetch --tags --force --progress -- https://github.com/dannya123-blm/inventory-mgmt-api.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision cc9cd6d84b88ff794b20acd56c57a42191031571 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f cc9cd6d84b88ff794b20acd56c57a42191031571 # timeout=10
Commit message: "changed json file for collection and changed url in postman to base url for 8001 port connections because jenkins ports were colliding on port 8000"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Check Tools)
[Pipeline] bat
```

This stage shows Jenkins successfully connecting to the GitHub repository and retrieving the latest version of the project. The console output confirms that the Jenkinsfile was obtained directly from the repository and that the correct branch and commit were checked out. The logs also show Git commands executing without errors, which confirms that the pipeline has access to the repository and can pull updates when changes are made.

Figure 2: Tool check output

This stage confirms that all required tools are installed and accessible within the Jenkins environment. The output shows Python, Docker, Node.js, and Newman versions. Newman installation is also confirmed. This ensures the environment is correctly configured before continuing.

```
C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline>python --version
Python 3.11.0
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline>docker --version
Docker version 29.3.0, build 5927d80
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline>node --version
v24.13.1
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline>npm install newman

up to date, audited 149 packages in 1s

21 packages are looking for funding
  run `npm fund` for details

15 vulnerabilities (6 moderate, 8 high, 1 critical)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline>npx newman --version
6.2.2
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Generate README)
[Pipeline] bat
```

Figure 3: README generation output

This stage confirms that all required tools are installed and accessible within the Jenkins environment. The output shows Python, Docker, Node.js, and Newman versions. Newman

installation is also confirmed. This ensures the environment is correctly configured before continuing.

```
C:\ProgramData\Jenkins\.jenkins\workspace\InventoryAPI-Pipeline>python generate_readme.py
README.txt generated successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Docker Image)
[Pipeline] bat
```

Figure 4: Docker build output

This stage shows the Docker image being built successfully using the project source code. The console output confirms that the Dockerfile was loaded, dependencies were installed, and all application files were copied into the container environment. Each build step completed without errors, and the final image was exported and tagged as `inventory-`

api. The successful completion of this stage confirms that the application was packaged correctly and is ready to run inside a container.

```
C:\ProgramData\Jenkins\jenkins\workspace\InventoryAPI-Pipeline>docker build -t inventory-api .
#0 building with "default" instance using docker driver

#1 [internal] load build definition from dockerfile
#1 transferring dockerfile: 469B 0.0s done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/ubuntu:22.04
#2 DONE 0.8s

#3 [internal] load .dockerignore
#3 transferring context: 180B done
#3 DONE 0.0s

#4 [1/6] FROM docker.io/library/ubuntu:22.04@sha256:ce4a593b4e323dcc3dd728e397e0a866a1bf516a1b7c31d6aa06991baec4f2e0
#4 resolve docker.io/library/ubuntu:22.04@sha256:ce4a593b4e323dcc3dd728e397e0a866a1bf516a1b7c31d6aa06991baec4f2e0 0.0s done
#4 DONE 0.1s

#5 [internal] load build context
#5 transferring context: 620.37kB 0.7s done
#5 DONE 0.7s

#6 [2/6] WORKDIR /app
#6 CACHED

#7 [3/6] RUN apt-get update && apt-get install -y python3 python3-pip curl && rm -rf /var/lib/apt/lists/*
#7 CACHED

#8 [4/6] COPY requirements.txt .
#8 CACHED

#9 [5/6] RUN pip3 install --no-cache-dir -r requirements.txt
#9 CACHED

#10 [6/6] COPY . .
#10 DONE 3.8s

#11 exporting to image
#11 exporting layers
#11 exporting layers 3.7s done
#11 exporting manifest sha256:18b3708a48ac8fb7e028e05b0873fdb4eb976230869e6279fe11c9768da24c9f 0.0s done
#11 exporting config sha256:f17e920bc964872ec8a77006729aa034e0d0ad981b0e3bdab5a28d9b809d7ad0 0.0s done
#11 exporting attestation manifest sha256:70fdb0e0689a26afdde7e6959721214fb9383c34ca6583d25a4fd53dcade04cc2 0.0s done
#11 exporting manifest list sha256:446e511caff5df2bd566f0ffdeb64c8b08b34ec206bd111b7307fce28ae52682 0.0s done
#11 naming to docker.io/library/inventory-api:latest done
#11 unpacking to docker.io/library/inventory-api:latest
#11 unpacking to docker.io/library/inventory-api:latest 3.0s done
#11 DONE 6.9s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Docker Container)
rsync --rsync --rsync
```

Figure 6: API startup check

This stage shows Jenkins verifying that the API is running before executing automated tests. A PowerShell script sends repeated requests to the /docs endpoint on port 8001

until a successful response is received. The script checks for a status code of 200, which confirms that the FastAPI application has started correctly and is ready to handle requests. This step prevents test failures by ensuring the API is available before Newman tests begin.

```
C:\ProgramData\Jenkins\jenkins\workspace\InventoryAPI-Pipeline>powershell -Command "$maxAttempts=20; for($i=1; $i -le $maxAttempts; $i++){ try { $response = Invoke-WebRequest -Uri 'http://localhost:8001/docs' -UseBasicParsing; if($response.StatusCode -eq 200){ exit 0 } } catch { Start-Sleep -Seconds 3 } exit 1"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Newman Tests)
[Pipeline] bat
```

Figure 7: Newman test execution

This output shows the execution of all API tests using Newman. Each endpoint is called and validated using predefined assertions. The results confirm that all requests return status code 200, responses are valid JSON, and the expected data is returned. The summary shows 7 requests executed and 27 assertions completed with zero failures. This confirms that all endpoints function correctly under automated testing.

First image:

Shows individual API requests being executed with successful status codes and valid responses.

```
[Pipeline] { (Run Newman Tests)
[Pipeline] bat

C:\ProgramData\Jenkins\jenkins\workspace\InventoryAPI-Pipeline>npx newman run .postman\InventoryAPI.postman_collection.json --env-var baseUrl=http://localhost:8001
(node:23624) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
(Use `node --trace-deprecation ...` to show where the warning was created)
newman
```

Second image:

Shows detailed validation checks for each endpoint, confirming correct data structure and behavior.

InventoryAPI

```
→ Get All
GET http://localhost:8001/getAll [200 OK, 27.98kB, 166ms]
✓ Status code is 200
✓ Response is JSON
✓ Response is an array
✓ Array is not empty

→ getSingleProduct
GET http://localhost:8001/getSingleProduct?id=1001 [200 OK, 267B, 14ms]
✓ Status code is 200
✓ Response is JSON
✓ Has ProductID
✓ Correct product returned

→ Add New Product
POST http://localhost:8001/addNew [200 OK, 165B, 36ms]
✓ Status code is 200
✓ Response is JSON
✓ Product added successfully

→ Delete Product
DELETE http://localhost:8001/deleteOne?id=9997 [200 OK, 167B, 20ms]
✓ Status code is 200
✓ Response is JSON
✓ Delete successful

→ Starts With S
GET http://localhost:8001/startsWith?letter=s [200 OK, 3.49kB, 37ms]
✓ Status code is 200
✓ Response is JSON
✓ Response is array
✓ All names start with letter

→ Paginate
GET http://localhost:8001/paginate?start_id=1001&end_id=1025 [200 OK, 1.57kB, 16ms]
✓ Status code is 200
✓ Response is JSON
✓ Response is array
✓ Max 10 results

→ Convert Price
GET http://localhost:8001/convert?id=1008 [200 OK, 178B, 505ms]
✓ Status code is 200
✓ Response is JSON
✓ Has ProductID
✓ Has USD price
✓ Has EUR price
```

Third image:

Shows the final test summary with all requests and assertions completed successfully with zero failures.

	executed	failed
iterations	1	0
requests	7	0
test-scripts	7	0
prerequisite-scripts	0	0
assertions	27	0
total run duration: 1371ms		
total data received: 32.94kB (approx)		
average response time: 113ms [min: 14ms, max: 505ms, s.d.: 167ms]		

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Create ZIP Artifact)
[Pipeline] bat
```

Figure 8: ZIP creation output

This output shows the creation of the final project archive within the pipeline. The system generates a timestamped ZIP file which includes all required project files such as source code, Docker configuration, Postman collection, and the generated README. The process completes successfully, confirming that the packaging stage works as expected.

```
C:\ProgramData\Jenkins\jenkins\workspace\InventoryAPI-Pipeline>powershell -Command "$timestamp = Get-Date -Format 'yyyy-MM-dd-HH-mm-ss'; $zipName = 'complete-' + $timestamp + '.zip'; Compress-Archive -Path app.py,database.py,models.py,load_csv.py,requirements.txt,Dockerfile,.dockerignore,generate_readme.py,README.txt,.postman -DestinationPath $zipName -Force"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] bat

C:\ProgramData\Jenkins\jenkins\workspace\InventoryAPI-Pipeline>docker rm -f inventory-api-container 1>nul 2>&1 || exit /b 0
[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints
[Pipeline] }
[Pipeline] // stage
```

Figure 9: Cleanup and final success

This output shows the creation of the final project archive within the pipeline. The system generates a timestamped ZIP file which includes all required project files such as source code, Docker configuration, Postman collection, and the generated README. The process completes successfully, confirming that the packaging stage works as expected.

```
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Final Build Artifacts (ZIP File)

This output shows the final ZIP artifacts generated by the Jenkins pipeline. The first image confirms that the pipeline successfully created timestamped ZIP files and stored them as build artifacts. The second image shows the contents of the ZIP file, including all required project files such as app.py, database.py, models.py, Dockerfile, requirements.txt, and README.txt. The third image shows the Postman collection stored inside the archive,

which is used for automated testing. These results confirm that the pipeline correctly packages all components into a single downloadable file.

InventoryAPI-Pipeline



Last Successful Artifacts

-  [complete-2026-03-30-16-13-46.zip](#) 5.96 KiB [view](#)
-  [complete-2026-04-01-18-32-23.zip](#) 5.93 KiB [view](#)
-  [README.txt](#) 1.16 KiB [view](#)

complete-2026-03-30-16-13-46.zip - ZIP archive, unpacked size 19,962 bytes						
Name	Size	Packed	Type	Modified	CRC32	
..			File folder			
.postman	11,745	1,492	File folder			
.dockerignore	138	93	DOCKERIGNORE F...	26/03/2026 23:...	E036CB2A	
app.py	3,993	1,146	Python.File	26/03/2026 23:...	BDCC05...	
database.py	311	214	Python.File	26/03/2026 23:...	FDA9E0CC	
Dockerfile	430	289	File	26/03/2026 23:...	313C20A1	
generate_readm...	1,387	629	Python.File	26/03/2026 23:...	EE8D781C	
load_csv.py	482	308	Python.File	26/03/2026 23:...	34072EF9	
models.py	162	116	Python.File	26/03/2026 23:...	ECEF3534	
README.txt	1,184	513	Text Document	30/03/2026 16:...	896FEBAC	
requirements.txt	130	102	Text Document	26/03/2026 23:...	D03E3E0A	

complete-2026-03-30-16-13-46.zip\postman - ZIP archive, unpacked size 19,962 bytes						
Name	Size	Packed	Type	Modified	CRC32	
..			File folder			
InventoryAPI.po...	11,392	1,258	JSON File	26/03/2026 23:...	458D9933	
resources.yaml	353	234	Yaml Source File	26/03/2026 23:...	0E463EA5	

API Performance Monitoring

This output shows the monitoring system tracking API uptime and response performance. The results confirm that the API remains available with consistent status code responses and stable response times. This demonstrates that the deployed service is reliable and performs consistently under monitoring checks.

The screenshot displays the Sentry Uptime Monitor interface. At the top, there's a navigation bar with 'Alert Rules' and 'History' tabs. Below this, a filter bar shows 'My Teams' (1), 'node', and 'Alert Type: All'. A search bar is also present. The main content area lists alert rules. One rule is highlighted: 'inventory api - /getAll', which is 'Actively monitoring every 5 minutes'. To the right of this rule, there's a green checkmark, a 'node' icon, a 'TD' icon, and a menu icon.

Below the alert rules, there's a detailed view for the 'inventory api - /getAll' rule. It shows a '1h' time range and a status bar with green indicators. A message states: 'No issues relating to this uptime alert have been found.' Below this, a 'Checks List' table is shown with columns: ID, TIMESTAMP, STATUS, HTTP CODE, DURATION, REGION, and TRACE. The table contains 10 rows of data, all with a status of 'Uptime' and an HTTP code of 200. To the right of the table, there's a 'Checked URL' section showing the URL 'https://unready-clemente-acronically.ng' and a legend for 'Uptime' (green circle), 'Failure' (red circle), and 'Downtime' (red circle). Below the legend, there's a 'Configuration' section with settings: Check Interval (Every 5 minutes), Timeout (After 5.00 seconds), Failure tolerance (3 failure checks), Recovery tolerance (1 up check), Environment (/getAll), and Owner (TD).

ID	TIMESTAMP	STATUS	HTTP CODE	DURATION	REGION	TRACE
bff3a271	Mar 26, 10:39 PM GMT+0:45 PM	Uptime	200	149ms	Netherlands	d87229ed 7 spans
174b1239	Mar 26, 11:31 PM GMT	Uptime	200	141ms	Germany	8e861a78 7 spans
e95b2e55	Mar 26, 11:26 PM GMT	Uptime	200	304ms	EU West	460e7af6 7 spans
7197a216	Mar 26, 11:21 PM GMT	Uptime	200	499ms	Netherlands	75e7bb90 7 spans
4cb2b2ca	Mar 26, 11:16 PM GMT	Uptime	200	492ms	Germany	4bc908b6 7 spans
5b39ef23	Mar 26, 11:11 PM GMT	Uptime	200	208ms	EU West	89ce88ea 7 spans
7b9db27d	Mar 26, 11:06 PM GMT	Uptime	200	151ms	Netherlands	dc05050c 7 spans
f69caa9c	Mar 26, 11:01 PM GMT	Uptime	200	151ms	Germany	a0056f23 7 spans
2d4cf943	Mar 26, 10:56 PM GMT	Uptime	200	163ms	EU West	a6b48154 7 spans

Conclusion

This project demonstrates the development of a complete Inventory Management API using FastAPI, covering core backend and DevOps practices. The API supports key operations including retrieval, creation, deletion, filtering, pagination, and currency conversion. Each endpoint returns structured JSON responses with correct HTTP status codes, ensuring consistency and reliability.

Testing was implemented using Postman and Newman to validate all endpoints. Results confirm that all requests executed successfully, with every assertion passing and no

failures recorded. This verifies correct functionality and consistent behavior across repeated runs.

Docker was used to containerise the application, ensuring a stable and portable runtime environment with all dependencies included. Jenkins automated the full workflow, including source code retrieval, environment checks, Docker build, container execution, API testing, and artifact generation. This reflects a structured Continuous Integration and delivery process with repeatable and efficient execution.

The pipeline generated a final ZIP artifact containing all required project files, confirming successful automation and completeness. Monitoring results showed stable uptime and consistent response performance, indicating reliable system operation.

This project demonstrates how combining API development, automated testing, containerization, and pipeline automation results in a robust, scalable, and production-ready system aligned with modern software engineering standards.

References

- Pahl, C. (2015) 'Containerization and the PaaS cloud', *IEEE Cloud Computing*, 2(3), pp. 24–31
- Bass, L., Weber, I. and Zhu, L. (2015) *DevOps: A Software Architect's Perspective*. Boston: Addison-Wesley
- Newman, S. (2015) *Building Microservices*. Sebastopol: O'Reilly Media
- Zhang, Q., Chen, M., Li, L., Wu, D. and Luo, Y. (2018) 'A survey on cloud computing: state-of-the-art and research challenges', *Journal of Internet Services and Applications*, 9(1), pp. 1–21

Humble, J. and Farley, D. (2011) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston: Addison-Wesley

Fielding, R. T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis. University of California, Irvine

Documentation (supporting sources)

FastAPI Documentation. Available at: <https://fastapi.tiangolo.com/>

Docker Documentation. Available at: <https://docs.docker.com/>

Jenkins User Documentation. Available at: <https://www.jenkins.io/doc/>

Postman Learning Center. Available at: <https://learning.postman.com/>

A significant portion of the concepts and approaches applied in this project were informed by the module lecture notes, which provided guidance on DevOps practices, API development, testing, and deployment.